

install_certificate

REFERENCE

API consumer

install a certificate or key into the reader's trust store

Overview

`install_certificate` **installs a certificate (or key) into the reader's trust store** — for MQTT-TLS brokers, Wi-Fi Enterprise (EAP), the file store, or client/server auth. Content is supplied one of two ways, chosen by `certDetails.certSource`:

- `HTTP` (default) — the device **downloads** each component from the URLs in `url[]`.
- `DIRECT` — you provide the content **inline** in `certificateBundle`.

It is a state-changing command on the device-management (MGMT) endpoint. Installed certificates are then referenced by name from `config_endpoint` (`MQTT_TLS`) and `set_wifi` (Enterprise).

`install_certificate` **is asynchronous: the reply `code 1` is an accept, not an install confirmation.** Verify with `get_installed_certificates` (and, for `HTTP`, the `FILE_DOWNLOAD` alerts). Do not gate success on `code == 0`. See [Lifecycle](#).

Details

Property	Value
Interaction Pattern	Asynchronous Request / Accept (device acks with code 1; completion verified out-of-band)
State Change	STATE-CHANGING (writes to the trust store; asynchronous)
Reversibility	REVERSIBLE (remove with <code>delete_certificate</code>)
Required Request Fields	<code>command</code> , <code>requestId</code> , <code>certDetails</code> (with <code>type</code> and <code>authenticationType</code>)
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
API Version	<code>v1.21</code> (API Version Reporting)
Related	<code>get_installed_certificates</code> , <code>delete_certificate</code> , <code>config_endpoint</code> , <code>set_wifi</code>

When to Use

Use `install_certificate` to provision the certificates other commands depend on:

- **Before MQTT-TLS** — install the `ca_cert` / `client_cert` / `client_key` (type `mqtt`) that `config_endpoint` references in `securityParams`.
- **Before Wi-Fi Enterprise** — install `wifi`-type certificates that `set_wifi` references in its `certificate` array.
- **File-store trust** — install a `filestore` CA used to authenticate HTTPS downloads.
- **Choose the source:** use `DIRECT` when you can embed the content (no network dependency); use `HTTP` when the device can reach your file server (watch the `FILE_DOWNLOAD` alerts).

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/MGMT/clients/cmnd/<serial></code>	On the management endpoint.
Response	Reader → App	<code><tenantId>/MGMT/clients/resp/<serial></code>	The <code>code 1</code> accept. Correlated by the in-payload <code>requestId</code> .
Event	Reader → App	<code><tenantId>/MGMT/clients/event/<serial></code>	For <code>HTTP</code> installs, the device emits asynchronous <code>FILE_DOWNLOAD</code> alerts (per-file download/save) here. See alerts .

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

See [Topics and Channels](#).

Key Fields

- `certDetails.certSource` (request) — `HTTP` (use `url`) or `DIRECT` (use `certificateBundle`).
- `certDetails.type` (request) — the certificate category (`mqtt`, `wifi`, `filestore`, ...); scopes where the certificate can be used.

- `certDetails.name` (request) — the logical name you choose for the entry. It is optional at install time, but `name` together with `type` is what identifies an installed certificate afterward: `delete_certificate` takes exactly that pair, and a delete that omits `name` removes every installed certificate of the matching `type`.
- `certDetails.authenticationType` (request) — `NONE` or `CERTIFICATE` (certificate-authenticated download).
- `certificateBundle.client_key` / `cert_key_password` (request) — ⚠ secrets you send inline (DIRECT); mask them. In `HTTP` mode a `url` entry with `key: cert_key_password` likewise carries a plaintext password in its `value`. See the credential warning below.
- `response.code` (response) — `1` means *accepted*, not *installed*; confirm with `get_installed_certificates`. See [Response Codes](#).

Request Payload

The envelope is `command`, `requestId`, and the `certDetails` payload. Pick a **certificate scenario** below; the Example and Schema update together. Use the **Schema** tab for the full field reference and the **Example** tab for the concrete payload.

⚠ YOU TRANSMIT PRIVATE KEYS

In `DIRECT` mode `certificateBundle.client_key` (private key) and `cert_key_password` (password) are secrets in the request — mask them as `*****`. In `HTTP` mode, a `url` entry with `key: cert_key_password` also carries a plaintext password in its `value` — treat it the same way. Send `install_certificate` only over secured channels. The device never echoes them, and the read-back never exposes private-key material.

Certificate scenario

MQTT certificates with certificate authentication



Request payload

Example: mqtt · HTTP · CERTIFICATE auth

```
1      {
2        "command" : "install_certificate" ,
3        "requestId" : "cert-test-001" ,
4        "certDetails" : {
5          "type" : "mqtt" ,
6          "name" : "testMqttCert" ,
7          "authenticationType" : "CERTIFICATE" ,
8          "certSource" : "HTTP" ,
9          "verificationType" : "VERIFY_HOST_AND_PEER" ,
10         "-----BEGIN CERTIFICATE-----
MIIDPTCCAiWgAwIBAgIUHtj1WDceNR9Nf+vZBB1Ne7s
BQAwLjEPMA0GA1UEAwwGVGVzdENBMQ4wDAYDVQQKDAV
VVMwHhcNMjYwNTEwMTMxNDQ3WhcNMzYwNTA4MTMxNDQ
ZXN0Q0ExDjAMBgNVBAoMBVplYnJhMQswCQYDVQQGEwJ
AQEBBQADggEPADCCAQoCggEBANR8E/mxOQ5uTPW/muW
dMLIQ+TVgb9mcOz80VnpgkQqomhbZaIG4CtnBufyIJv
lGoPC4M469Rm+0oEN9GkUHgtgF2N4Qt9bYomySux3yy
XE/a7XWdXeJkpIB0Mzzf7MoNq14AfX431RKgvpMgAqZ
aKIAJV2avJd87dF8x+K80kLBErDnKtuNFpXN7KoC7BB
eL9MY5Ik7POmR0AM/TxRBLAXyoZHSPdEpEoaQZjSzJ7
VR00BBYEFJHms5XPEyFYnVcf2++X9N8UlkahMB8GA1U
nVcf2++X9N8UlkahMA8GA1UdEwEB/wQFMAMBAf8wDQY
AL4HEEMhuQ9sQLFE9wiGw4BIILXnX30FbphWM8/xpHK
+QUhaU9hruBEajT7mYjKvZ9NtUumcAmAh3s29KD4Npj
zCmqlAqGVzK3ve/Fuurgdbk3kApOZTTyqFw629dmmRh
Y6ZtwwAJygC4/WCfWg/7XaeQ+6kqKtY6ijSJFFCNwEP
rvm5UUPRMGCs20JOucntSF8uPOICNWUER3DbqAJ4jXK
Vqh5mX6k2PZe0eylcukyu/s=
-----END CERTIFICATE-----"
        "caCertificateFileContent" :
        "url" : [
          {
            "key" : "ca_cert" ,
            "value" : "https://192.168.0.104:8443/mqtt_ca_cert.pem"
          },
          {
            "key" : "client_cert" ,
            "value" : "https://192.168.0.104:8443/mqtt_client_cert.pem"
          },
          {
            "key" : "client_key" ,
            "value" : "https://192.168.0.104:8443/mqtt_client_key.pem"
          }
        ]
      }
    }
```

Request payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
1	comm and *	string	—	install_certificate
2	requ estI *	string	—	abc123
3	cert Deta *	object	—	—
4	nam e	string	—	sotiAgent
5	ty pe *	enum	client server mqtt wifi filestore	client
6	au th en ti ca *	enum	NONE CERTIFICATE	CERTIFICATE
7	cer tSo urc e	enum	HTTP DIRECT	HTTP

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
8	certFormat	enum	PEM BASE64	BASE64
9	url	array	—	—
10	key	enum	ca_cert client_key client_cert cert_key_password	ca_cert
11	value	string	—	https://192.168.0.104:8443/mqtt_pem
12	verificationType	enum	VERIFY_NONE VERIFY_HOST_AND_PEER	VERIFY_NONE

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
13	certificateBundle	object	—	—
14	certificate	string	—	-----BEGIN CERTIFICATE-----...
15	clientIdentity	string	—	LS0tLS1CRUdJTiBSU0Eg...
16	clientCertificate	string	—	LS0tLS1CRUdJTiBDRVJU...

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
17	cert_key_password	string	—	—
18	caCertificateContent	string	—	-----BEGIN CERTIFICATE-----...

Request Semantics

Modes. `certDetails.certSource` selects how the certificate content is supplied — `HTTP` (download) or `DIRECT` (inline):

- `HTTP` (**default**). Supply `url[]` with a `{key, value}` entry per component (`ca_cert`, `client_cert`, `client_key`, `cert_key_password`). The device downloads each; for `authenticationType: CERTIFICATE` over HTTPS, supply `caCertificateFileContent` (or rely on an installed filestore CA). Requires **device** → **file-server reachability**; missing `url` returns code `29`. The device emits `FILE_DOWNLOAD` alerts as it fetches.
- `DIRECT`. Supply `certificateBundle` with inline PEM content for one `type`. No network dependency; missing `certificateBundle` returns code `30`.

Invalid enum values (e.g. `type`, `certSource`) return code `23`. The contract does not forbid unrecognized fields, so a misspelled field still passes schema validation. How the reader responds to such a field is not specified in the available sources, so check field names client-side before you send.

- **Choosing `type`**. The enum accepts `client`, `server`, `mqtt`, `wifi`, and `filestore`. Current deployments need `wifi` (Enterprise EAP, referenced by `set wifi`), `filestore` (the CA that authenticates HTTPS downloads), and `mqtt` (broker TLS, referenced by `config endpoint`). `client` and `server` are the generic client-side and server-side categories; the schema gives them no further gloss beyond "the certificate category that identifies where the certificate is used", and its own field examples pair `type: client` with an agent certificate (`name: sotiAgent`).
- **One install, one `type` — and file-store trust must be typed `filestore`**. `certDetails.type` is a single string with a five-member enum, not a list, and `certificateBundle` supplies certificate content "for one certificate `type`", so each `install_certificate` call installs exactly one category. A file-store CA is therefore typed `filestore` and cannot also be typed `client` or `server` — send a second `install_certificate` if you need a second category. This matters for HTTPS downloads: with `authenticationType: CERTIFICATE` and an empty `caCertificateFileContent`, the schema names an already-installed `filestore` certificate as the trust anchor the device falls back to, so installing the same CA under `client` or `server` leaves no `filestore` entry for that fallback to use.
- **`type` and `authenticationType` are required but not firmware-enforced**. The schema requires both within `certDetails`, but the device accepts a payload omitting them (still returns `code 1`) and installs a malformed entry (e.g. `type: null`). Always send both; do not rely on the device to reject their absence.
- **Certificate size limit — 4 KB**. Certificate content is capped at **4 KB (4096 bytes) per certificate file**, not per request; keep each certificate within the limit.
- **PEM, or base64-encoded PEM**. The device supports certificates in **PEM** format, or as PEM text base64-encoded again (`certFormat: BASE64`) — PEM being the base64 encoding delimited by header/footer markers, e.g. `-----BEGIN CERTIFICATE-----` / `-----END CERTIFICATE-----`, `-----BEGIN RSA PRIVATE KEY-----` / `-----END RSA PRIVATE KEY-----`, and `-----BEGIN ENCRYPTED PRIVATE KEY-----` / `-----END ENCRYPTED PRIVATE KEY-----`.
- **PKCS1 (RSA) key encoding**. The reader requires private keys in **PKCS1** format — the public-key-cryptography standard for RSA.

Response Payload

The immediate reply is an **accept** — `command`, `requestId`, `apiVersion`, and `response` with `code 1`. It confirms receipt, not installation.

Result

Certificate installed



Response payload

Example: accepted (code 1)

```
1  {
2    "command" : "install_certificate",
3    "requestId" : "18996",
4    "apiVersion" : "V1.1",
5    "response" : {
6      "code" : 1,
7      "description" : "Command payload is accepted"
8    }
9  }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	install_certificate	Echoes the executed command.
2	requestId *	string	—	abc123	Echoes the request's requestId — pairs reply to request.
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1	API version reported by this command. Emitted per-handler, not device-wide.
4	response *	object	—	—	Accept/status envelope. This is an asynchronous accept, not a terminal install result (see below).

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
5	c o d e *	integer	0–101	0	1 = accepted (the success/accept code for this command — not 0). Non-1 = rejected. See Response Codes below.
6	d e s c r i p t i o n *	string	—	Success	Human-readable status.

Lifecycle

Acknowledgment

Because the accept (`code 1`) is not a terminal result, confirm the install actually landed. There is **no** terminal success/failure message on the response topic after the `code 1` accept. A consumer that treats `code == 0` as the success condition will misread every `install_certificate` reply.

Progress

For `HTTP`, watch `FILE_DOWNLOAD` **alerts** on the event topic — per-file download/save progress. These confirm download + save only, not trust-store persistence; still verify with the read-back.

Confirmation

Read back with `get installed certificates` — the installed components appear in `installedCerts.certInfo[]` (named `{name}_{component}`). This is the authoritative confirmation. (Note: private keys are never listed, and serials read back truncated to 8 bytes.)

Response Codes

`install_certificate` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
1	Command payload is accepted (<code>IOT_STATUS_CMD_PAYLOAD_ACCEPTED</code>) — the success/accept code for this command . The install proceeds asynchronously; verify with <code>get_installed_certificates</code> .
0	Success — terminal success; in practice <code>install_certificate</code> acks with 1, and 0 is what the <code>get_installed_certificates</code> read-back returns.
9	File not found (<code>IOT_STATUS_FILE_NOT_FOUND</code>) — a referenced file/URL could not be found.
21	Certificate not found (<code>IOT_ERROR_CERT_NOT_FOUND</code>) — the named certificate is absent.
23	Invalid enum value (<code>IOT_ERROR_INVALID_ENUM</code>) — a field (e.g. <code>type</code> , <code>certSource</code> , <code>authenticationType</code> , <code>verificationType</code>) is outside its allowed set.
29	Url missing for HTTP source (<code>IOT_ERROR_URL_MISSING</code>) — <code>certSource: HTTP</code> (or defaulted) with no/empty <code>url</code> .
30	Certificate content missing for direct source (<code>IOT_ERROR_CERT_CONTENT_MISSING</code>) — <code>certSource: DIRECT</code> with no <code>certificateBundle</code> .
34	Certificate base64 decode failed (<code>IOT_ERROR_CERT_BASE64_DECODE_FAILED</code>) — the base64-encoded certificate content could not be decoded. Re-encode it and resend.
36	Missing client key or client certificate (<code>IOT_MISSING_CLIENT_KEY_OR_CERT</code>) — <code>client_cert</code> and <code>client_key</code> were not sent together. Always install a client certificate with its key.
101	Default error — generic catch-all.

Examples

Publish `install_certificate`; the device acks with the `code 1` envelope under [Response Payload](#). Every contract request example is in the **Certificate scenario** dropdown under [Request Payload](#), and the **Result** dropdown under Response Payload shows the accepted reply and the two source-specific errors (`29` for a missing `url`, `30` for missing `certificateBundle` content). The examples below show what the dropdowns do not.

A **DIRECT** **install** embeds the content inline — the private key and any key password are masked:

Request — DIRECT (inline, secrets masked)

```
1  {
2    "command" : "install_certificate" ,
3    "requestId" : "cert-test-003" ,
4    "certDetails" : {
5      "type" : "mqtt" ,
6      "name" : "testMqttCert" ,
7      "authenticationType" : "NONE" ,
8      "certSource" : "DIRECT" ,
9      "verificationType" : "VERIFY_HOST_AND_PEER" ,
10     "certificateBundle" : {
11       "-----BEGIN CERTIFICATE-----
12         "ca_cert" : ... ,
13         "-----END CERTIFICATE-----"
14       "client_cert" : ... ,
15         "-----END CERTIFICATE-----"
16       "client_key" : "*****" ,
17         "-----END RSA PRIVATE KEY-----"
18     "cert_key_password" : "XXXXXXXXX"
19   }
20 }
```

A minimal **filestore** **HTTP install** downloads a single CA:

Request — filestore (HTTP, single CA)

```
1  {
2    "command" : "install_certificate" ,
3    "requestId" : "19001" ,
4    "certDetails" : {
5      "name" : "root_certs" ,
6      "type" : "filestore" ,
7      "authenticationType" : "NONE" ,
8      "certSource" : "HTTP" ,
9      "verificationType" : "VERIFY_HOST_AND_PEER" ,
10     "url" : [
11       {
12         "key" : "ca_cert" ,
13         "value" : "https://192.0.2.107:8080/filestore_ca_cert.pem"
14       }
15     ]
16   }
17 }
```

Notes and Caveats

Installing a certificate leaves the active MDM/MGMT control session intact. After installing, reference the certificate by its logical `name` from `config_endpoint` or `set_wifi`. For the concept-level view, see The Security Model and Install and Manage Certificates.

Related:  [The Security Model](#) ·  [get installed certificates](#) ·  [delete certificate](#) ·  [config_endpoint](#) ·  [set_wifi](#) ·  [Response Codes](#)

install_certificate

2 of 3